Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 4

За седьмой семестр

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Хомиченко И.А.

Проверил:

Никитин С.К.

2026

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Вариант:

-

Вариант 7 (c0 = −4, c1 = −9)

A	B	A ⊕ B
−4	−4	−4
−4	−9	−9

−9	−4	−9
−9	−9	−4

7	3.0	0.22	1.5	0.55	8	6

Задачи лабораторной работы

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.
2. Нормализовать ряд по минимуму и максимуму **обучающего участка**: x_norm = (x - x_min) / (x_max - x_min). Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. **Объяснить**, почему min/max нельзя брать по всему ряду.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p (вход – p значений, цель – следующее значение). Указать количество получившихся примеров в каждой выборке.
4. Реализовать многослойный персептрон p–N–1:
    - скрытый слой – сигмоида;
    - выходной слой – линейный (задача регрессии), f(s) = s;
    - функция потерь – MSE;
    - обучение – онлайн-режим, перемешивание порядка примеров в каждой эпохе;
    - вывести формулы градиентов для линейного выходного слоя и сигмоидного скрытого слоя и привести их в отчёте.

5. Критерий остановки: средняя по обучающей выборке ошибка $E_s \le E_e$, где $E_e = 10^{-4}$ (нормализованная шкала), либо достижение 20 000 эпох. Скорость обучения подобрать самостоятельно из диапазона 0.01–0.3 и обосновать выбор.
6. **Базовый эксперимент.** Обучить сеть с параметрами p и N из своего варианта. Зафиксировать число эпох, итоговую E_s, а также метрики на обучающем и тестовом участках **в исходной шкале**: MAE, RMSE и MAPE (если ряд проходит через ноль – вместо MAPE использовать sMAPE, и пояснить почему).
7. **Исследование окна.** При фиксированном N из варианта обучить сеть для $p \in \{2, 4, p_{вар}, 2 \cdot p_{вар}\}$. Для каждого p выполнить по 5 запусков с разными seed и свести в таблицу: среднее и разброс числа эпох, среднее и разброс тестового MAE, число запусков, не достигших критерия остановки.

8. **Исследование скрытого слоя.** При фиксированном p из варианта обучить сеть для N ∈ {1, 2, N_вар, 2·N_вар}. Выполнить ту же серию из 5 запусков и заполнить аналогичную таблицу.

9. **Многошаговый прогноз.** Для лучшей по тестовому MAE конфигурации выполнить рекурсивный прогноз на 30 шагов вперёд, начиная с k = 140: каждое предсказание подставляется в окно как входное значение для следующего шага. Построить график «эталон – рекурсивный прогноз» и график абсолютной ошибки в зависимости от номера шага. Определить, на каком шаге ошибка превышает 5 % от размаха ряда.

10. Построить графики:
    - ряд с границей train/test и одношаговым прогнозом сети на обоих участках на одних осях;
    - сходимость (Es от эпохи) для базовой конфигурации и для двух конфигураций из исследования окна на одних осях (логарифмическая шкала по Y);
    - диаграмма рассеяния «эталон – прогноз» на тестовом участке с диагональю y = x;
    - рекурсивный прогноз и рост ошибки по шагам (п. 9).

11. Реализовать режим функционирования: пользователь вводит p последовательных значений ряда в исходной шкале (либо номер k, с которого брать окно из ряда). Программа выводит:
    - нормализованные входы;
    - прогноз в нормализованной шкале (4 знака после запятой);
    - прогноз в исходной шкале;
    - истинное значение x(k) и абсолютную ошибку (если окно взято из ряда).

Продемонстрировать работу не менее чем на 3 окнах из обучающего участка, 3 окнах из тестового участка и 1–2 пользовательских окнах (в том числе с «выбросом»).

12. Вставить в отчет фото/скан рукописных расчётов. Вверху листа от руки должны быть написаны Ваша Фамилия, Инициалы, номер группы, вариант и дата выполнения:
    - Графическая схема пайплайна (Блок-схема). Составьте корректную цепочку обработки данных, используя предложенный набор блоков (приложение 1). Объясните свой выбор.
    - Ручной расчёт 1 шага обучения. На отдельном листе бумаги от руки распишите один полный шаг (Forward + Backward pass) для сети 3–2–1 со следующими параметрами, привязанными к вашему варианту V:
      - Входы: X = [0.2, 0.5, 0.8], Цель: d = 0.6
      - Веса скрытого слоя: W11 = 0.05*V, W12 = -0.2, W21 = 0.3, W22 = 0.05*V, W31 = -0.1, W32 = 0.4 (b1=0.1, b2=-0.1)
      - Веса выходного слоя: W_out1 = 0.5, W_out2 = -0.2*V (b_out = 0.2)

13. Сделать вывод: ответить на все вопросы ниже и проанализировать результаты.

Код программы:

```
from pathlib import Path
import csv
import numpy as np
import matplotlib.pyplot as plt

VARIANT = 7
A, B, C, D = 3.0, 0.22, 1.5, 0.55
P_VARIANT, N_VARIANT = 8, 6
LR = 0.12
EE = 1e-4
MAX_EPOCHS = 20000
SEEDS = [0, 1, 2, 3, 4]
SHOW_PLOTS = False
INTERACTIVE = True
DIR = Path(__file__).resolve().parent / 'results'
DIR.mkdir(exist_ok=True)


def series(a=A, b=B, c=C, d=D, length=200):
    k = np.arange(length, dtype=float)
    return a * np.cos(b * k) + c * np.sin(d * k)


def normalize(seq, low, high):
    return (np.asarray(seq) - low) / (high - low)


def denormalize(values, low, high):
    return np.asarray(values) * (high - low) + low


def windows(seq, p, target_indices):
    xs, ys = [], []
    for k in target_indices:
        if k < p:
            continue
        xs.append(seq[k-p:k][::-1])  # x[k-1], x[k-2], ..., x[k-p]
        ys.append(seq[k])
    return np.array(xs, dtype=float), np.array(ys, dtype=float).reshape(-1,
1)


def sigmoid(z):
    return 1. / (1. + np.exp(-np.clip(z, -50, 50)))


def initialize(p, n, seed):
    rng = np.random.default_rng(seed)
    return [rng.uniform(-0.2, 0.2, (p, n)), np.zeros(n),
            rng.uniform(-0.2, 0.2, n), 0.0]


def forward(x, model):
    w1, b1, w2, b2 = model
    h = sigmoid(x @ w1 + b1)
    out = h @ w2 + b2
    return h, out


def predict(X, model):
    return forward(np.asarray(X), model)[1].reshape(-1)
```

```
def train(X, y, p, n, seed, lr=LR, max_epochs=MAX_EPOCHS, threshold=EE):
    model = initialize(p, n, seed)
    rng = np.random.default_rng(seed + 50000)
    hist = []
    # Es = среднее (y_pred - y)^2 по обучающей выборке
    for ep in range(max_epochs + 1):
        preds = predict(X, model)
        es = float(np.mean((preds - y.ravel()) ** 2))
        hist.append(es)
        if es <= threshold or ep == max_epochs:
            break
        for idx in rng.permutation(len(X)):
            x, target = X[idx], float(y[idx, 0])
            h, output = forward(x, model)
            error = float(output) - target
            # dL/dz_out = 2*(output-target) для MSE и линейного выхода
            delta_out = 2. * error
            # использовать СТАРЫЕ выходные веса при расчете delta_hidden
            delta_hidden = delta_out * model[2] * h * (1. - h)
            model[2] -= lr * delta_out * h
            model[3] -= lr * delta_out
            model[0] -= lr * np.outer(x, delta_hidden)
            model[1] -= lr * delta_hidden
    return {'model': model, 'epochs': ep, 'es': hist[-1],
            'history': np.array(hist), 'converged': hist[-1] <= threshold,
            'seed': seed, 'p': p, 'n': n}


def metrics(true, pred):
    true, pred = np.asarray(true), np.asarray(pred)
    mae = float(np.mean(np.abs(pred - true)))
    rmse = float(np.sqrt(np.mean((pred - true)**2)))
    denom = np.abs(true) + np.abs(pred)
    smape = float(100 * np.mean(np.divide(2*np.abs(pred-true), denom,
                        out=np.zeros_like(denom), where=denom > 1e-12)))
    return {'MAE': mae, 'RMSE': rmse, 'sMAPE': smape}


def dataset(seq, p, low, high):
    norm = normalize(seq, low, high)
    # Целевые индексы определяют train/test; тест может использовать
исторический train-контекст.
    trX, trY = windows(norm, p, range(p, 140))
    teX, teY = windows(norm, p, range(140, 200))
    return norm, trX, trY, teX, teY


def run_one(seq, low, high, p, n, seed):
    norm, trX, trY, teX, teY = dataset(seq, p, low, high)
    item = train(trX, trY, p, n, seed)
    for key, X, y in [('train', trX, trY), ('test', teX, teY)]:
        pred = denormalize(predict(X, item['model']), low, high)
        true = denormalize(y.ravel(), low, high)
        item[key] = metrics(true, pred)
    return item


def unique_values(values):
    return list(dict.fromkeys(values))


def summary(runs):
    epochs = np.array([r['epochs'] for r in runs], float)
```

```
    errors = np.array([r['test']['MAE'] for r in runs], float)
    return {'epochs_mean': epochs.mean(), 'epochs_range': np.ptp(epochs),
            'mae_mean': errors.mean(), 'mae_range': np.ptp(errors),
            'failed': sum(not r['converged'] for r in runs)}


def save_chart(fig, name):
    fig.savefig(DIR/name, dpi=165, bbox_inches='tight')
    if SHOW_PLOTS:
        plt.show()
    plt.close(fig)


def recursive(model, norm, start=140, steps=30):
    p = model['p']
    context = list(norm[start-p:start])
    result = []
    for _ in range(steps):
        value = float(predict(np.array([context[-p:][::-1]]),
model['model'])[0])
        result.append(value)
        context.append(value)
    return np.array(result)


def visualizations(seq, low, high, baseline, window_runs, best, norm):
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(range(200), seq, label='Ряд, эталон', linewidth=1.8)
    for name, idxs in [('Одношаговый train', range(baseline['p'], 140)),
                       ('Одношаговый test', range(140, 200))]:
        X, _ = windows(norm, baseline['p'], idxs)
        ax.plot(list(idxs), denormalize(predict(X, baseline['model']), low,
high),
                label=name, linestyle='--')
    ax.axvline(139.5, color='black', linestyle=':', label='Граница 70/30')
    ax.legend(fontsize=8); ax.set(xlabel='k', ylabel='x(k)', title='Ряд и
одношаговые прогнозы')
    save_chart(fig, '01_series_and_predictions.png')

    fig, ax = plt.subplots(figsize=(9, 4))
    variants = [baseline]
    for p in unique_values([2, 4] + list(window_runs)):
        cand = window_runs.get(p, [])
        if cand and p != baseline['p']:
            variants.append(cand[0])
        if len(variants) == 3:
            break
    for run in variants:
        ax.plot(run['history'], label=f'p={run["p"]}, N={run["n"]},
seed={run["seed"]}')
    ax.axhline(EE, color='black', linestyle=':', label='Ee=1e-4')
    ax.set_yscale('log'); ax.set(xlabel='Эпоха', ylabel='Es (MSE)',
title='Сходимость обучения')
    ax.legend(fontsize=8); save_chart(fig, '02_convergence.png')

    Xte, yte = windows(norm, baseline['p'], range(140, 200))
    true = denormalize(yte.ravel(), low, high)
    pred = denormalize(predict(Xte, baseline['model']), low, high)
    fig, ax = plt.subplots(figsize=(6, 5))
    ax.scatter(true, pred, s=19, alpha=.7)
    bound = [min(true.min(), pred.min()), max(true.max(), pred.max())]
    ax.plot(bound, bound, 'k--', label='y=x')
    ax.set(xlabel='Эталон', ylabel='Прогноз', title='Тест: эталон и прогноз')
    ax.legend(); save_chart(fig, '03_scatter.png')
```

```
    rec = denormalize(recursive(best, norm), low, high)
    actual = seq[140:170]
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(range(140, 170), actual, label='Эталон', marker='.')
    ax.plot(range(140, 170), rec, label='Рекурсивный прогноз', marker='.')
    ax.legend(); ax.set(xlabel='k', ylabel='x(k)', title=f'30 шагов,
p={best["p"]}, N={best["n"]}')
    save_chart(fig, '04_recursive.png')
    abs_error = np.abs(actual - rec)
    limit = 0.05 * (seq.max() - seq.min())
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(range(1, 31), abs_error, marker='o', markersize=3)
    ax.axhline(limit, color='red', linestyle='--', label='5 % размаха ряда')
    ax.legend(); ax.set(xlabel='Шаг прогноза', ylabel='Абсолютная ошибка',
title='Рост ошибки')
    save_chart(fig, '05_recursive_error.png')
    exceeds = np.flatnonzero(abs_error > limit)
    return (int(exceeds[0]+1) if len(exceeds) else None), float(limit)


def operation_examples(model, seq, low, high):
    norm = normalize(seq, low, high)
    p = model['p']
    print('\nРЕЖИМ ФУНКЦИОНИРОВАНИЯ: 3 TRAIN + 3 TEST + 2 пользовательских
окна')
    examples = [p+4, p+12, 120, 140, 155, 190]
    print('k | прогноз norm | прогноз x | истина | |ошибка|')
    for k in examples:
        inp = norm[k-p:k][::-1]
        y_n = predict(np.array([inp]), model['model'])[0]
        y = denormalize(y_n, low, high)
        print(f'{k:3d} | {y_n:12.4f} | {y:9.5f} | {seq[k]:7.5f} | {abs(y-
seq[k]):.5f}')
    for label, vals in [('обычное', seq[40-p:40]), ('с выбросом',
np.r_[seq[40-p:39], seq[39]+3])]:
        inp = normalize(vals[::-1], low, high)
        y_n = predict(np.array([inp]), model['model'])[0]
        print(f'{label}: входы_norm={np.round(inp, 4)},
прогноз_norm={y_n:.4f}, '
              f'прогноз_x={denormalize(y_n, low, high):.6f}, истина=не
задана')


def interactive_loop(model, seq, low, high):
    p = model['p']; norm = normalize(seq, low, high)
    print(f'\nВведите k от {p} до 199, или {p} чисел в исходной шкале через
пробел; q — выход.')
    while True:
        raw = input('Окно или k: ').strip()
        if raw.lower() == 'q': return
        try:
            parts = raw.replace(';', ' ').split()
            if len(parts) == 1:
                k = int(parts[0]); assert p <= k <= 199
                values = seq[k-p:k][::-1]; true = seq[k]
            else:
                assert len(parts) == p
                values = np.array([float(v.replace(',', '.')) for v in
parts])
                true = None
            nx = normalize(values, low, high)
            pr_n = float(predict(np.array([nx]), model['model'])[0])
            pr = float(denormalize(pr_n, low, high))
```

```
            print(f'Входы_norm: {np.round(nx,4)}')
            print(f'Прогноз_norm: {pr_n:.4f}; прогноз_исходный: {pr:.6f}')
            if true is not None:
                print(f'Истина x({k})={true:.6f}; абсолютная ошибка={abs(pr-
true):.6f}')
        except (ValueError, AssertionError):
            print('Некорректный ввод. Укажите индекс k или ровно p чисел.')


def main():
    print('Параметры из таблицы варианта 7.')
    seq = series(); low, high = seq[:140].min(), seq[:140].max()
    print(f'Вариант V={VARIANT};
a={A},b={B},c={C},d={D},p={P_VARIANT},N={N_VARIANT}')
    print(f'min_train={low:.6f}, max_train={high:.6f}, lr={LR}, Ee={EE}')
    print(f'Размер выборок: train={140-P_VARIANT}; test=60')
    baseline = run_one(seq, low, high, P_VARIANT, N_VARIANT, 0)
    print(f'\nБАЗА: эпох={baseline["epochs"]}, Es={baseline["es"]:.8f}, '
          f'сошлась={baseline["converged"]}')
    for section in ['train', 'test']:
        print(section, ', '.join(f'{name}={value:.6f}' for name,value in
baseline[section].items()))
    # Кэш исключает повторное обучение совпадающих конфигураций
    cache = {(P_VARIANT, N_VARIANT, 0): baseline}
    def get(p, n, seed):
        key = p, n, seed
        if key not in cache:
            cache[key] = run_one(seq, low, high, p, n, seed)
        return cache[key]
    win = {p: [get(p,N_VARIANT,seed) for seed in SEEDS]
           for p in unique_values([2,4,P_VARIANT,2*P_VARIANT])}
    hidden = {n: [get(P_VARIANT,n,seed) for seed in SEEDS]
              for n in unique_values([1,2,N_VARIANT,2*N_VARIANT])}
    for title, results in [('ОКНО', win), ('СКРЫТЫЙ СЛОЙ', hidden)]:
        print('\n' + title)
        print('значение | epochs_mean | epochs_range | test_MAE_mean |
test_MAE_range | не сошлось')
        for value, runs in results.items():
            s = summary(runs)
            print(f'{value:8} | {s["epochs_mean"]:11.1f} |
{s["epochs_range"]:12.0f} | '
                  f'{s["mae_mean"]:13.6f} | {s["mae_range"]:14.6f} |
{s["failed"]}')
    with (DIR/'all_runs.csv').open('w',newline='',encoding='utf-8-sig') as f:
        writer = csv.writer(f,delimiter=';')

writer.writerow(['p','N','seed','epochs','converged','Es','MAE_train','RMSE_t
rain','sMAPE_train',
                         'MAE_test','RMSE_test','sMAPE_test'])
        for run in cache.values():

writer.writerow([run['p'],run['n'],run['seed'],run['epochs'],int(run['converg
ed']),run['es'],
                 *[run['train'][v] for v in ['MAE','RMSE','sMAPE']],
                 *[run['test'][v] for v in ['MAE','RMSE','sMAPE']]])
    best = min(cache.values(), key=lambda item: item['test']['MAE'])
    print(f'\nЛУЧШАЯ:
p={best["p"]},N={best["n"]},seed={best["seed"]},MAE={best["test"]["MAE"]:.6f}
')
    first_exceed, five_percent = visualizations(seq, low, high, baseline,
win, best, normalize(seq,low,high))
    print(f'Порог 5% размаха: {five_percent:.6f}; первое превышение:
{first_exceed or "нет за 30 шагов"}')
    operation_examples(baseline, seq, low, high)
```

```
    print('Графики и all_runs.csv:', DIR)
    if INTERACTIVE:
        interactive_loop(baseline, seq, low, high)

if __name__ == '__main__':
    main()
```

Результат программы:

Вариант V=7
a=3.0, b=0.22, c=1.5, d=0.55
p=8, N=6
min_train=-4.499515
max_train=4.051066
lr=0.12, Ee=0.0001
Размер выборок:
train=132; test=60
БАЗА:
эпох=90
Es=0.00009268
сошлась=True
train:
MAE=0.066795
RMSE=0.082318
sMAPE=7.787566
test:
MAE=0.064604
RMSE=0.079907
sMAPE=6.410946

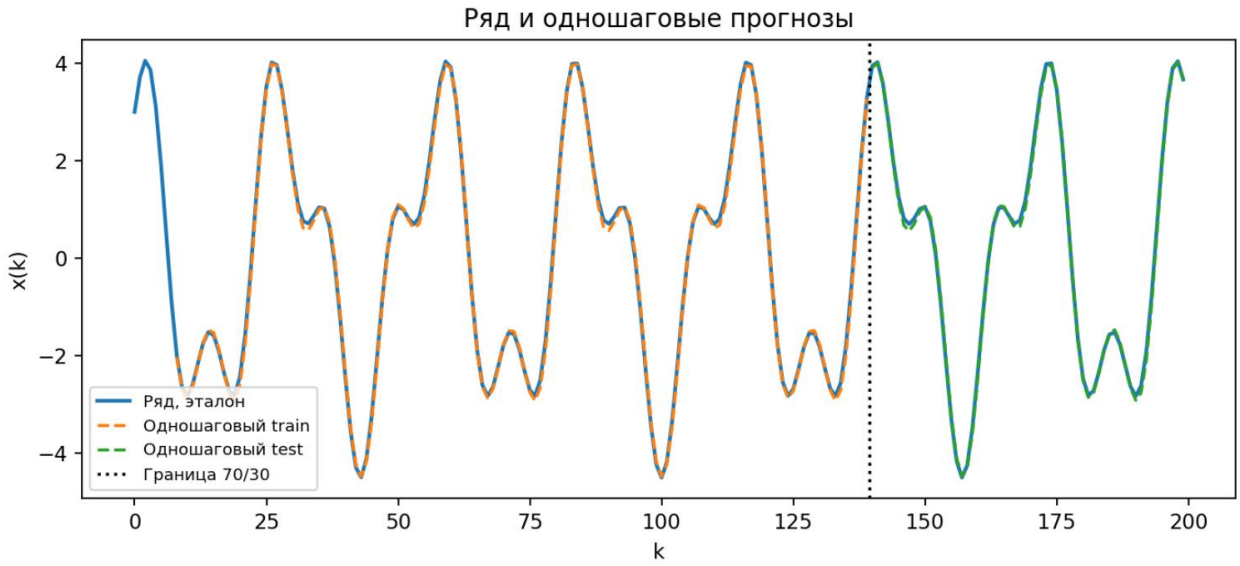
Исследование размера окна

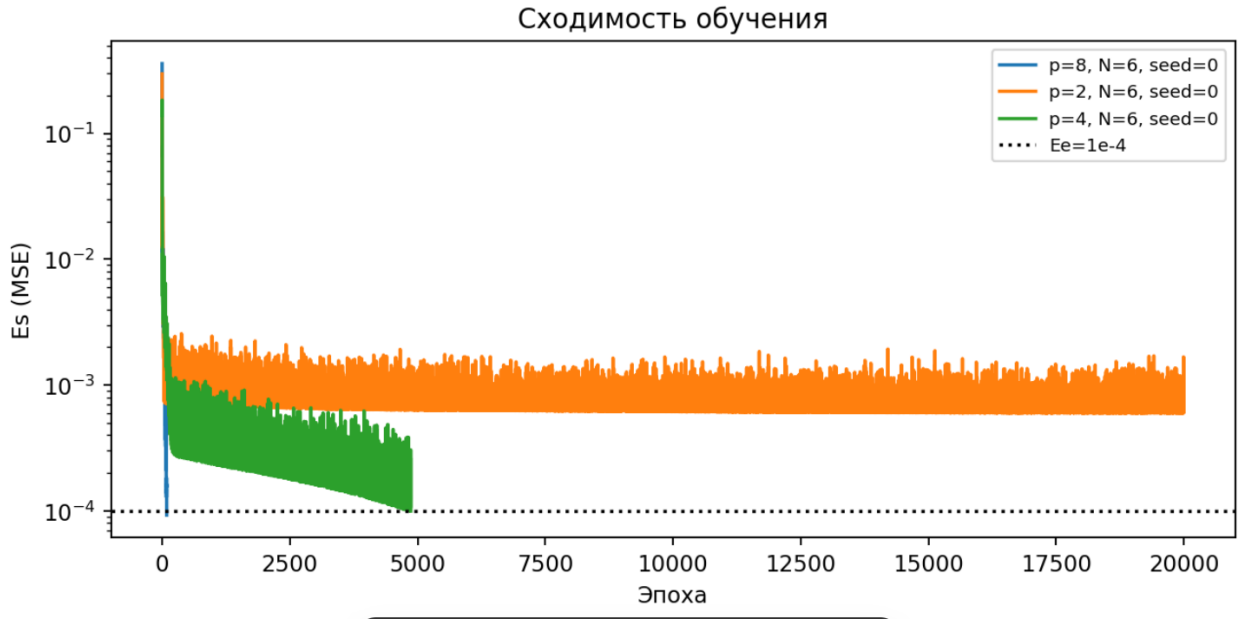
p	Среднее эпох	Средний MAE теста	Не сошлось
2	20 000	0,185056	5
4	4 982,2	0,067257	0
8	87,8	0,064850	0
16	10,2	0,054761	0

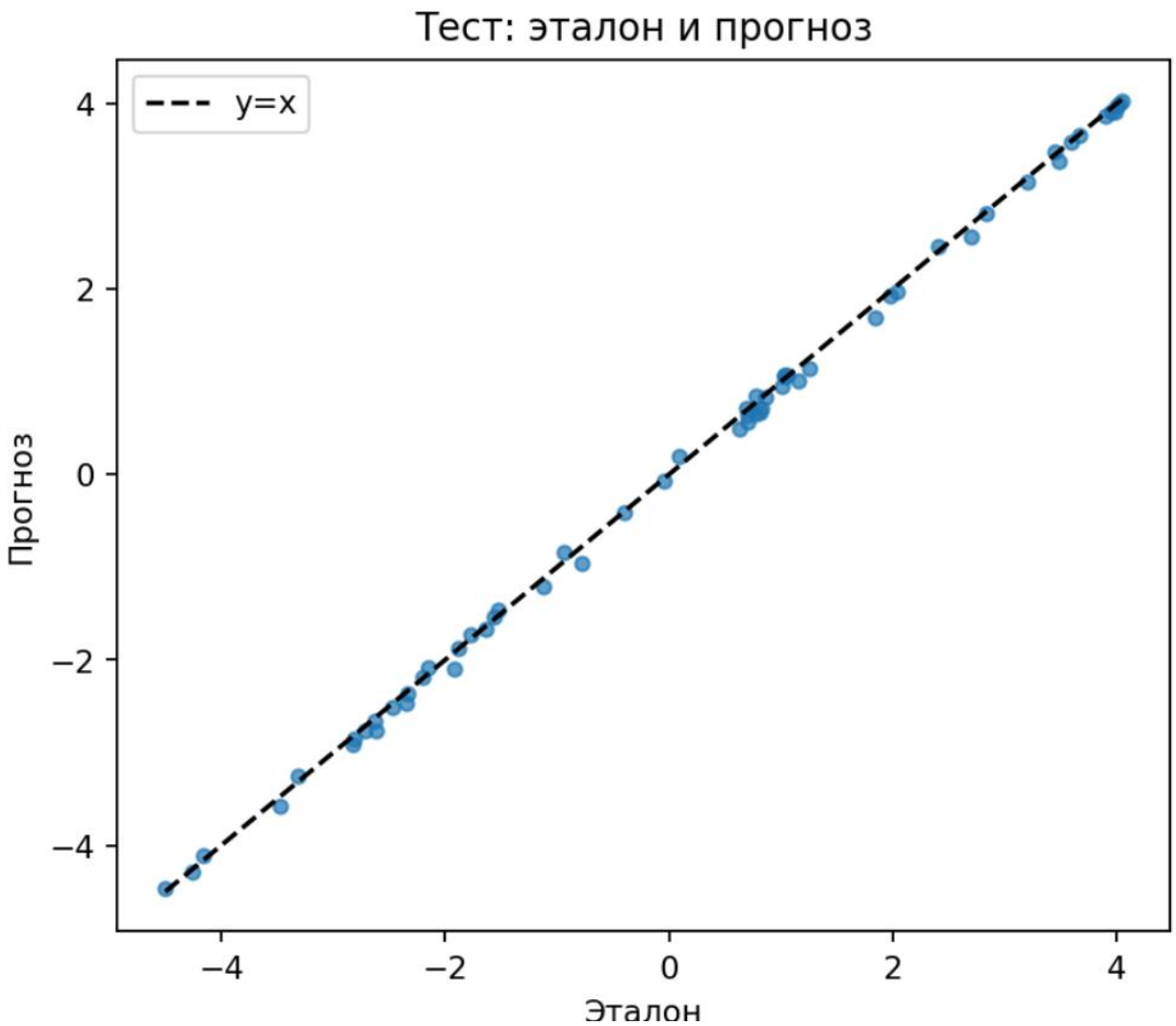
Исследование скрытого слоя

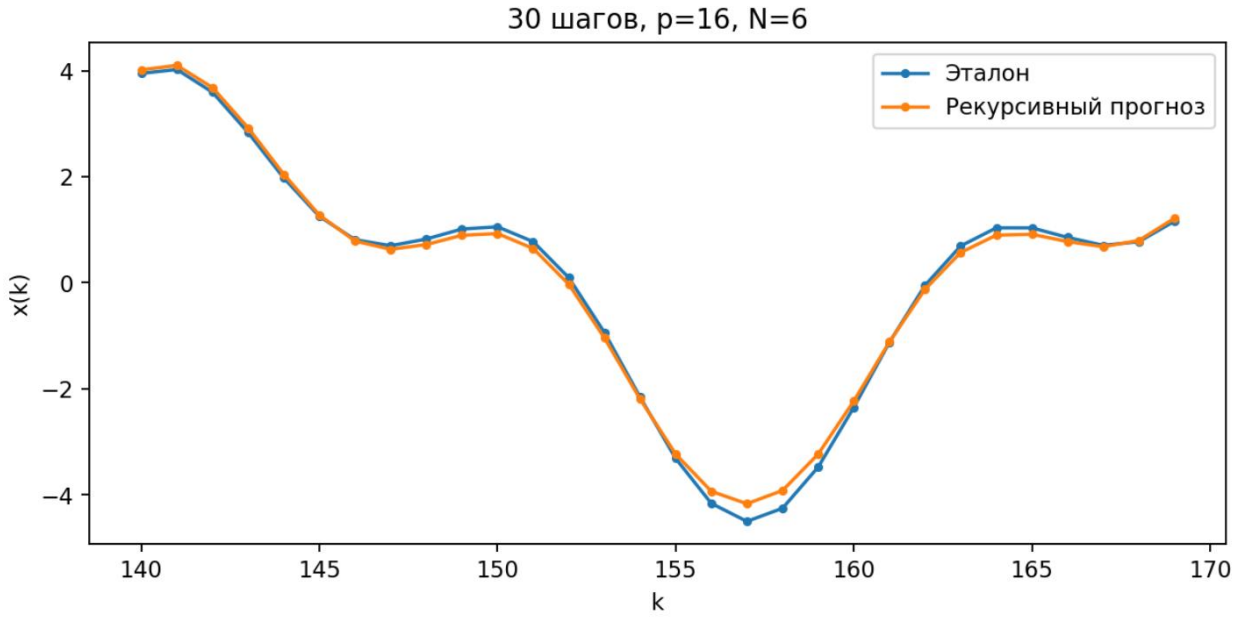
N	Среднее эпох	Средний MAE теста	Не сошлось
1	95,6	0,064629	0
2	90,8	0,066404	0
6	87,8	0,064850	0
12	92,8	0,065215	0

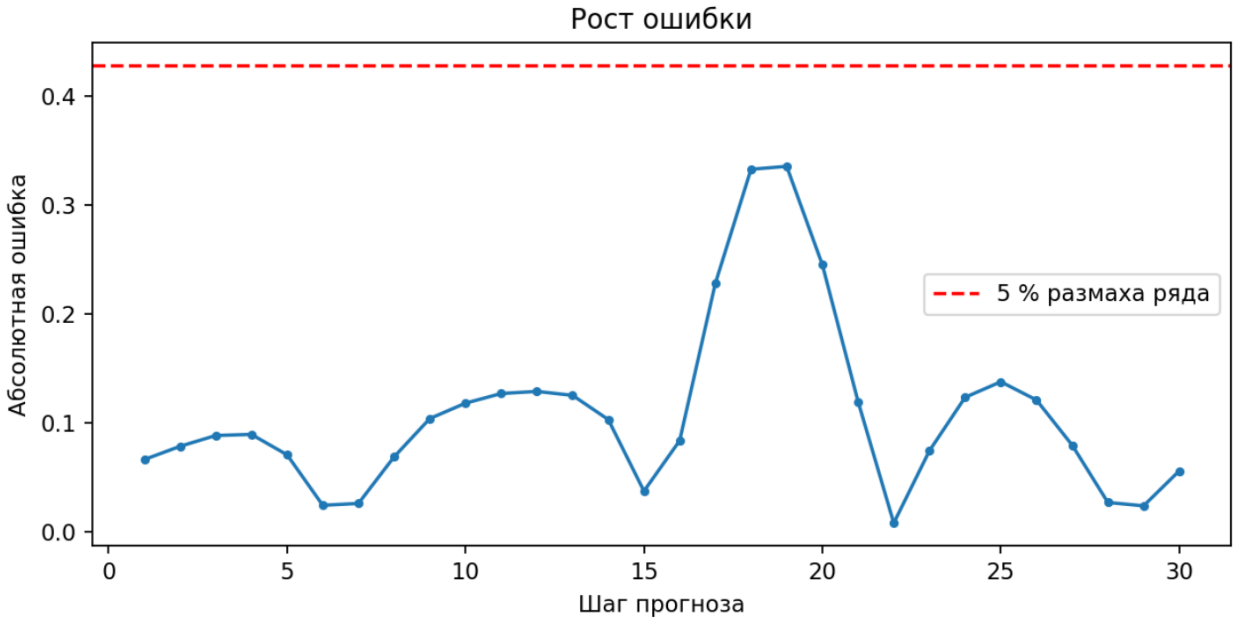
ЛУЧШАЯ:
p=16, N=6, seed=2
MAE=0.053276
Порог 5% размаха: 0.427529
Первое превышение: нет за 30 шагов


Ряд и одношаговые прогнозы
x(k)
k
Ряд, эталон
Одношаговый train
Одношаговый test
Граница 70/30


Сходимость обучения
Es (MSE)
Эпоха
p=8, N=6, seed=0
p=2, N=6, seed=0
p=4, N=6, seed=0
Ee=1e-4


Тест: эталон и прогноз
y=x
Прогноз
Эталон
4
2
0
−2
−4
−4
−2
0
2
4





В ходе лабораторной работы была реализована и обучена нейронная сеть MLP 8–6–1 для прогнозирования временного ряда по варианту 7. Обучение проводилось методом обратного распространения ошибки с использованием сигмоидальной функции на скрытом слое и линейной функции на выходе. Базовая модель достигла заданной ошибки за 90 эпох, показав MAE = 0,066795 на обучающей и MAE = 0,064604 на тестовой выборке. Проведено исследование влияния размера окна и количества скрытых нейронов на качество прогноза. Наилучший результат показала конфигурация с p = 16 и N = 6, достигнув тестовой MAE = 0,053276. При рекурсивном прогнозировании на 30 шагов ошибка не превысила 5% размаха ряда. Таким образом, нейронная сеть успешно справилась с прогнозированием временного ряда, а увеличение размера окна позволило повысить точность предсказаний.

Вывод: на основе многослойного персептрона p–N–1 решил задачу прогнозирования временного ряда. Научился формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество

одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.